Lezione 9 – Interfacce e package

`interface` e `implements`, più tipi per una classe, metodi `default` e `static`; `package`, `import` e visibilità

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

3 novembre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 8** (*15 min*)
   - uno di voi alla lavagna con `Data`, `Appuntamento` o `Anagrafe`: lo commentiamo insieme
2. **Le interfacce** (*25 min*)
   - `interface` e `implements`; l'interfaccia come tipo; più interfacce per una classe
   - metodi `default` e `static`; interfacce o classi astratte; finalmente `Cloneable`
3. **I package** (*20 min*)
   - `package`, nomi qualificati, cartelle; compilare ed eseguire; `import`
   - visibilità di package e riepilogo dei modificatori
4. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: un simulatore di dispositivi con due interfacce
   - *Insieme, passo passo*: lo stesso simulatore riorganizzato in package
5. **Wrap-up e Homework 9** (*10 min*)

Interfacce: il problema

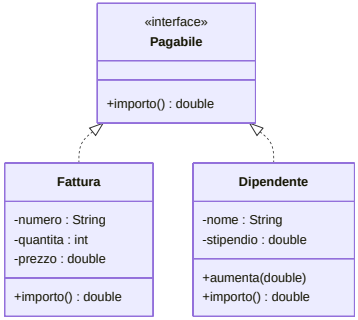
Una `Fattura` e un `Dipendente`: classi diverse, nessuna relazione *è-un* fra loro. Eppure la contabilità li tratta allo stesso modo: ognuno **va pagato**, di ognuno serve l'**importo**. Vorremmo un array con entrambi e un metodo `totale` che sommi gli importi.

Con l'ereditarietà (lezione 7), una superclasse astratta `Pagabile` con `importo()` astratto? Due problemi:

- `Pagabile` non avrebbe **nulla** da dare in eredità: niente variabili di istanza, costruttori, metodi con il corpo; solo l'intestazione di `importo()`
- una classe ha **una sola** superclasse: se `Dipendente` estendesse già `Persona`, non potrebbe estendere `Pagabile`

Interfaccia

Un'*interfaccia* (`interface`) è un tipo che elenca le intestazioni dei metodi che un oggetto offre, di norma senza corpo e sempre senza stato: dice **che cosa** si può chiedere a un oggetto, non **come** lo fa. Una classe che la *implementa* si impegna a scrivere quei metodi. È l'interfaccia della lezione 4 come tipo di Java.



In UML: `<<interface>>` sopra il nome; la freccia **tratteggiata** con il triangolo è la *realizzazione*: `Fattura` implementa `Pagabile`.

Dichiarare un'interfaccia – `Pagabile.java`

```java
/** Qualcosa che va pagato: ha un importo. */
public interface Pagabile {

    double importo();  // public abstract
}
```

- `interface` al posto di `class`; i metodi hanno solo l'**intestazione** (la firma più il tipo di risultato), chiusa dal punto e virgola, come i metodi astratti della lezione 7
- sono **implicitamente** `public abstract`: scriverlo è lecito ma ridondante. Niente `protected`: ciò che un'interfaccia offre è tutto pubblico
- può dichiarare **costanti**: `double TARIFFA = 2.0;` è implicitamente `public static final`, si usa come `Pagabile.TARIFFA`
- **non** può avere variabili di istanza né costruttori: non descrive uno stato, solo un comportamento
- sta nel file `Pagabile.java`: "una `public` per file, con il suo nome" vale anche per le interfacce

Che cosa contiene un'interfaccia

Intestazioni di metodi (astratti) e costanti. Da Java 8 anche metodi con il corpo, `default` e `static` (fra quattro slide). Mai variabili di istanza né costruttori.

	classe	**interfaccia**
parola chiave	`class`	`interface`
variabili di istanza	sì	no
costanti `static final`	sì	sì (implicito)
costruttori	sì	no
metodi con il corpo	sì	`default` e `static`
metodi astratti	solo se `abstract`	sì, è la norma
`new`	sì (se concreta)	no

Il nome è spesso un aggettivo: che cosa l'oggetto *sa fare*: `Pagabile`, `Stampabile`, `Cloneable`, `Comparable`.

Implementare un'interfaccia: `implements` – `Fattura.java`

```java
/** Una fattura: quantita' per prezzo. */
public class Fattura implements Pagabile {

    private final String numero;
    private final int quantita;
    private final double prezzo;

    public Fattura(String numero, int quantita,
                   double prezzo) {
        this.numero = numero;
        this.quantita = quantita;
        this.prezzo = prezzo;
    }

    @Override
    public double importo() {
        return quantita * prezzo;
    }
```

Non mostrato il `toString` (`"fattura " + numero`).
`Dipendente` è fatta allo stesso modo: `nome` e `stipendio`, costruttore, `importo()` che restituisce lo stipendio, `toString`, e in più `aumenta(double aumento)`.

```java
/** Un dipendente: va pagato lo stipendio. */
public class Dipendente implements Pagabile {
```

- `implements Pagabile` dopo il nome: la classe **promette** tutti i metodi dell'interfaccia. Se ne manca uno: *"Fattura is not abstract and does not override abstract method importo() in Pagabile"*: o lo si scrive, o la classe diventa `abstract`
- il metodo va dichiarato `public`: nell'interfaccia lo è; con meno visibilità, *"attempting to assign weaker access privileges"*. `@Override` vale anche qui
- il resto è libero: ogni classe ha le *sue* variabili di istanza, i suoi costruttori e altri metodi, come `aumenta`, che `Pagabile` non conosce

L'interfaccia come tipo: polimorfismo – `ProvaPagabili.java`

```java
/** LEGGE: la somma degli importi. */
public static double totale(Pagabile[] elenco) {
    double somma = 0;
    for (Pagabile p : elenco) {
        somma += p.importo();
    }
    return somma;
}

public static void main(String[] args) {
    Pagabile[] daPagare = {
        new Fattura("F-01", 3, 250),
        new Dipendente("Anna Russo", 1800),
        new Fattura("F-02", 1, 99.5)
    };
    for (Pagabile p : daPagare) {
        System.out.println(p + ": " + p.importo());
    }
    System.out.println(totale(daPagare));
```

```
fattura F-01: 750.0
dipendente Anna Russo: 1800.0
fattura F-02: 99.5
2649.5
```

L'interfaccia è un tipo

Variabili e array possono essere di tipo `Pagabile`, parametri compresi (variabili locali inizializzate dalla chiamata, lezione 3): un riferimento `Pagabile` porta a un oggetto di **qualunque** classe che la implementa (upcasting implicito, come con le superclassi).

- `p.importo()`: **binding dinamico** (lezione 7): la JVM esegue l'`importo` della classe dell'oggetto. `totale` vale anche per classi future
- attraverso `p` si vedono solo i metodi di `Pagabile` (e di `Object`): `p.aumenta(100)` non compila. Per usarlo: `p instanceof Dipendente`, poi il cast `(Dipendente) p`. `instanceof` vale anche per le interfacce
- `new Pagabile()` non compila: gli oggetti nascono dalle classi

Più interfacce per una classe – versione 2

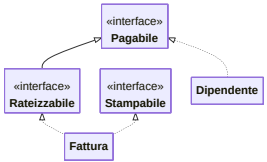
```java
/** Un pagabile che si puo' pagare a rate. */
public interface Rateizzabile extends Pagabile {

    int numeroRate();  // public abstract
```

```java
/** Una fattura: a rate e stampabile. */
public class Fattura
        implements Rateizzabile, Stampabile {
```

```java
@Override
public int numeroRate() {
    return rate;
}

@Override
public void stampa() {
    System.out.println("FATTURA n. " + numero);
    System.out.println("totale " + importo()
            + " in " + rate + " rate");
}
```



- `implements A, B`: la classe promette i metodi di **entrambe** (`Stampabile`: solo `stampa()`). Una sola superclasse, ma quanti tipi interfaccia vuole: `Fattura` è `Rateizzabile`, `Stampabile` e `Pagabile`: *ereditarietà multipla* di tipi, senza stato
- `extends` e `implements` insieme: `class C extends S implements A, B`
- un'interfaccia può **estendere** un'altra: chi implementa `Rateizzabile` scrive `importo()` **e** `numeroRate()`

Metodi `default` e `static` – `ProvaPagabili.java` (versione 2)

```java
public interface Pagabile {

    double importo();  // public abstract

    static double totale(Pagabile[] elenco) {
        double somma = 0;
        for (Pagabile p : elenco) {
            somma += p.importo();
        }
        return somma;
    }
}
```

```java
public interface Rateizzabile extends Pagabile {

    int numeroRate();  // public abstract

    default double rata() {
        return importo() / numeroRate();
    }
}
```

```java
Fattura f = new Fattura("F-01", 3, 250, 2);
Dipendente d = new Dipendente("Anna", 1800);

Pagabile[] conti = { f, d };
System.out.println(Pagabile.totale(conti));
System.out.println(f.rata());    // default
// System.out.println(d.rata()); // errore
// System.out.println(f.totale()); // errore
```

```
2550.0
375.0
```

- un metodo `default` ha il corpo e lo **ereditano** le classi che implementano l'interfaccia: `f.rata()` funziona senza che `Fattura` l'abbia scritto. Usa solo i metodi dell'interfaccia (non c'è stato); si può ridefinire. `d.rata()` non compila: `d` non è `Rateizzabile`
- un metodo `static` si chiama con il nome dell'interfaccia: `Pagabile.totale(conti)`. **Diversamente dalle classi** (lezione 6), `f.totale()` **non compila**: i metodi `static` di un'interfaccia non passano alle classi che la implementano

Interfacce o classi astratte? E finalmente `Cloneable`

	classe astratta (L7)	**interfaccia** (oggi)
stato	variabili di istanza	no, solo costanti
costruttori	sì, con `super(...)`	no
metodi con il corpo	tutti quelli che servono	solo `default` e `static`
quante per classe	una (`extends`)	quante si vuole (`implements`)
la relazione	*è-un*, con una parte comune da ereditare	*sa fare*: un comportamento

- **classe astratta** quando le sottoclassi condividono *stato e codice* (`Forma`: `colore`, `piuGrandeDi`); **interfaccia** per classi non imparentate con gli stessi metodi, o per dare più tipi a una classe
- un ADT (lezione 5) è una struttura dati incapsulata dalle sue operazioni: la sua *interfaccia* è l'elenco delle operazioni, il corpo la realizzazione. Un'`interface` di Java è proprio quell'elenco, separato dalle realizzazioni: nella lezione 12 la pila sarà un'`interface` realizzata da due classi, con un array e con una lista; il client non cambia

```
public interface Cloneable {
}
```

`Cloneable`: nessun metodo

Dalla lezione 8: `implements Cloneable` non obbliga a scrivere nulla, nemmeno `clone()`, che c'è già in `Object` (la classe lo ridefinisce `public` solo per offrirlo ai client). **Marca** la classe come copiabile: `Object.clone()` controlla se l'oggetto è `instanceof Cloneable`; se no, lancia `CloneNotSupportedException`: ecco il `throws` (lezione 10).

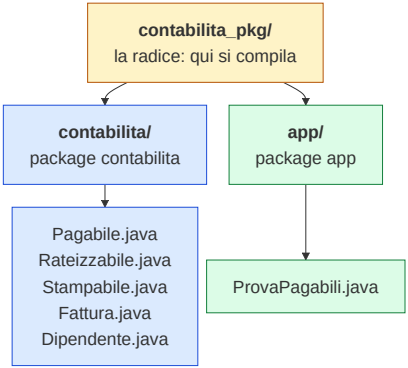
- `Cloneable` sta in `java.lang`, come `Object`: niente `import`
- lezione 12: `Comparable`, con `compareTo`, per `Arrays.sort` sui nostri oggetti

Package: il concetto – `contabilita_pkg/`

Package

Un *package* è un gruppo di classi e interfacce correlate, con un nome: è l'unità con cui Java organizza il codice in cartelle e librerie. `java.util` è il package con `Scanner`, `Arrays` e `Objects`; `contabilita` è il nostro.

- si dichiara con `package contabilita;` come **prima riga** del file (prima solo commenti); il file sta nella **cartella con lo stesso nome** del package: `contabilita/Fattura.java`
- il **nome qualificato** di una classe è package più classe: `contabilita.Fattura`; due classi `Persona` in package diversi convivono
- i package si **innestano**: `java.util.Arrays` è la classe `Arrays` nel package `util` dentro `java`. Nomi in minuscolo
- finora nessun file aveva la riga `package`: le classi senza package stanno nel **package di default**, senza nome, che `javac` e `java` cercano nella cartella corrente. Un package raggruppa classi coese: il *modulo* della lezione 4, su scala maggiore



Compilare ed eseguire con i package

In `contabilita/Fattura.java`, prima riga:

```
package contabilita;

/** Una fattura: a rate e stampabile. */
public class Fattura
        implements Rateizzabile, Stampabile {
```

Dal terminale, nella cartella **radice** `contabilita_pkg/`:

```
$ javac app/ProvaPagabili.java
$ java app.ProvaPagabili
2550.0
FATTURA n. F-01
totale 750.0 in 2 rate
```

Dentro app/, invece, **non** funziona:

```
$ cd app
$ java ProvaPagabili
Error: Could not find or load main class
ProvaPagabili
Caused by: java.lang.NoClassDefFoundError:
ProvaPagabili (wrong name: app/ProvaPagabili)
```

- ci si mette nella cartella **superiore** ai package, la radice del progetto
- `javac` riceve il **percorso** del file con il `main`. Come nella lezione 4, compila da solo anche le classi che servono, cercandole dalla cartella corrente: `contabilita.Fattura` in `contabilita/Fattura.java`. I `.class` finiscono accanto ai sorgenti
- `java` riceve il **nome qualificato** della classe: non un percorso, non `.class`. La JVM lo traduce in `app/ProvaPagabili.class` dalla cartella corrente: per questo da dentro app/ non lo trova (*wrong name*)
- finora tutte le nostre classi stavano nel package di default, lo stesso per tutte, e si vedevano con il nome semplice. Una classe del package di default, però, **non ha nome qualificato**: da app non si può né importare né nominare. Quindi, quando un progetto adotta i package, **tutte** le classi vanno in un package, anche quella con il `main`
- in VS Code basta aprire la cartella radice e usare *Run*

import: usare le classi di un altro package – app/ProvaPagabili.java

```java
package app;

import contabilita.Fattura;
import contabilita.Pagabile;

/** Il client, in un altro package. */
public class ProvaPagabili {

    public static void main(String[] args) {
        Fattura f = new Fattura("F-01", 3, 250, 2);
        contabilita.Dipendente d;   // qualificato
        d = new contabilita.Dipendente("Anna", 1800);
        Pagabile[] conti = { f, d };
        System.out.println(Pagabile.totale(conti));
        f.stampa();
    }
}
```

`Dipendente` non è importato: si usa con il nome qualificato. `Fattura`, invece, usa `Rateizzabile` e `Stampabile` senza `import`: le classi dello **stesso** package si vedono con il nome semplice.

import

Fuori dal suo package una classe si chiama con il nome qualificato; `import contabilita.Fattura;`, in testa al file, permette di scrivere solo `Fattura`. `import` non copia nulla: dice al compilatore **dove cercare**.

- tre forme: nome qualificato; `import` di una classe; `import contabilita.*;` per tutto il package
- classi con lo **stesso nome** in package diversi: se ne importa una sola; se servono entrambe, i due `import` sono un errore: la seconda si usa con il nome qualificato
- `java.lang` è importato **da solo**: `String`, `System`, `Math`, `Object`. `java.util` no: `Scanner`, `Arrays` e `Objects` li abbiamo sempre importati

Visibilità di package e riepilogo dei modificatori

membro	stessa classe	stesso package	sottoclasse in altro package	ovunque
`private`	sì	–	–	–
(nessuno)	sì	sì	–	–
`protected`	sì	sì	sì	–
`public`	sì	sì	sì	sì

Visibilità di package

Un membro **senza modificatore** è visibile da tutte le classi del suo package e da nessun'altra. Una **classe senza** `public` è visibile solo nel suo package: fuori non si può nominare, né importare. Il file non conta: conta il package.

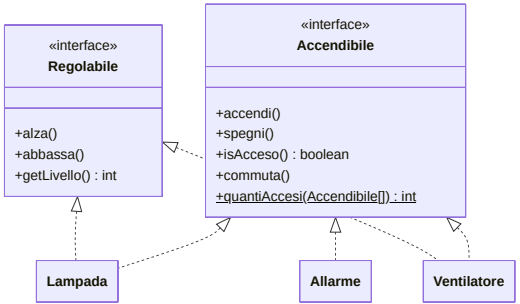
- il `protected` della lezione 7 è ora completo: sottoclassi **e** tutto il package
- per le classi solo due scelte: `public` (al più una per file, con il nome del file) o niente (visibile nel package): una classe scritta in un file non può essere `private`
- nelle interfacce ciò che si offre è `public`: costanti, metodi, e i metodi che le implementano
- la regola di sempre, completata: `private` per lo stato; `public` per l'interfaccia offerta ai client; **niente modificatore** per le classi e i metodi di servizio che devono vedere solo le classi dello stesso package; `protected` raramente

Commentiamo insieme (20 min) – il problema

Il problema. Un simulatore di casa domotica gestisce dispositivi diversi: lampade, ventilatori, un allarme. Il programma deve poterli **accendere e spegnere tutti insieme**, senza sapere di che tipo è ciascuno, e **regolare** quelli che si regolano: la luce di una lampada, la velocità di un ventilatore. L'allarme è solo acceso o spento. Le due interfacce, le tre classi e il `Simulatore` sono già scritti: li leggiamo riga per riga, cercando che cosa promette ogni interfaccia, chi implementa che cosa e dove serve `instanceof`.

Le regole

- ogni dispositivo si accende, si spegne, dice se è acceso e si può *commutare* (spegnere se acceso, accendere se spento)
- i dispositivi regolabili hanno un **livello** che si alza e si abbassa di un passo alla volta, solo se accesi, entro i limiti: la lampada da 1 a 10 (parte da 5), il ventilatore da 1 a 3 (parte da 1)
- l'allarme non è regolabile
- il programma stampa ogni dispositivo e quanti sono accesi



Commentiamo insieme – `Accendibile.java`

```java
/** Un dispositivo: si accende e spegne. */
public interface Accendibile {

    void accendi();

    void spegni();

    boolean isAcceso();

    /** default: inverte lo stato. */
    default void commuta() {
        if (isAcceso()) {
            spegni();
        } else {
            accendi();
        }
    }
```

```java
/** static: LEGGE quanti sono accesi. */
static int quantiAccesi(Accendibile[] elenco) {
    int n = 0;
    for (Accendibile d : elenco) {
        if (d.isAcceso()) {
            n++;
        }
    }
    return n;
}
```

- tre metodi astratti: ogni classe che implementa `Accendibile` li scriverà a modo suo
- `commuta` è `default`: scritto una volta sola in termini di `isAcceso`, `spegni` e `accendi`, che l'interfaccia non sa come sono fatti ma sa che ci saranno. Lo ereditano lampade, ventilatori e allarme
- `quantiAccesi` è `static`: lavora su un array di `Accendibile` qualsiasi e si chiama `Accendibile.quantiAccesi(...)`

Commentiamo insieme – `Regolabile.java` e `Lampada.java` (1/2)

```java
/** Ha un livello regolabile a passi. */
public interface Regolabile {

    void alza();

    void abbassa();

    int getLivello();
}
```

- le due interfacce sono indipendenti: un dispositivo può implementarne una sola o entrambe
- `Lampada` e `Ventilatore` le implementano entrambe, ma **non** sono imparentate: nessuna superclasse comune, ciascuna con le sue variabili di istanza (`luce`, `velocita`). Hanno in comune solo i due tipi interfaccia
- `Ventilatore` è fatto come `Lampada`: cambiano i nomi e i limiti (`velocita` da 1 a 3)

```java
/** Una lampada: luce regolabile da 1 a 10. */
public class Lampada
        implements Accendibile, Regolabile {

    private static final int MAX = 10;

    private final String stanza;
    private boolean accesa;
    private int luce;

    public Lampada(String stanza) {
        this.stanza = stanza;
        accesa = false;
        luce = 5;
    }

    @Override
    public void accendi() {
        accesa = true;
    }
```

Commentiamo insieme – Lampada.java (2/2) e Allarme.java

```
    @Override
    public void spegni() {
        accesa = false;
    }

    @Override
    public boolean isAcceso() {
        return accesa;
    }

    /** MODIFICA: piu' luce, se accesa. */
    @Override
    public void alza() {
        if (accesa && luce < MAX) {
            luce++;
        }
    }
```

I sei metodi promessi ci sono tutti, `public` e con `@Override`. Non mostrati: `abbassa()`, speculare ad `alza()`; `getLivello()`, che restituisce `luce`; `toString()`: `lampada cucina: accesa, luce 8/10` oppure `lampada cucina: spenta`.

```
/** Un allarme: acceso o spento. */
class Allarme implements Accendibile {

    private boolean attivo;

    @Override
    public void accendi() {
        attivo = true;
    }

    @Override
    public void spegni() {
        attivo = false;
    }

    @Override
    public boolean isAcceso() {
        return attivo;
    }
```

`Allarme` è dichiarata **senza** `public`: finché tutto sta nella stessa cartella non cambia nulla (ci torniamo fra poco). Non mostrati: il costruttore, che mette `attivo` a `false`, e `toString` (`allarme: attivo` o `allarme: disattivo`).

Commentiamo insieme – Simulatore.java

```
Accendibile[] casa = {
    new Lampada("cucina"),
    new Ventilatore("studio"),
    new Allarme(),
    new Lampada("camera")
};
for (Accendibile d : casa) {
    d.accendi();
}
for (Accendibile d : casa) {
    if (d instanceof Regolabile) {
        Regolabile r = (Regolabile) d;
        r.alza();
        r.alza();
        r.alza();
    }
}
casa[2].commuta();       // allarme: si spegne
casa[3].spegni();
for (Accendibile d : casa) {
    System.out.println(d);
}
System.out.println("accesi: "
        + Accendibile.quantiAccesi(casa));
```

```
lampada cucina: accesa, luce 8/10
ventilatore studio: acceso, velocita 3/3
allarme: disattivo
lampada camera: spenta
accesi: 2
```

Rispondiamo insieme:

1. perché `casa` può contenere lampade, ventilatori e allarmi, che non hanno una superclasse comune?
2. nel secondo ciclo, perché `d.alza()` non compilerebbe? E a che cosa servono `instanceof` e il cast?
3. il ventilatore riceve tre `alza()` e arriva a 3, non a 4; la lampada a 8: dove sta la differenza?
4. `Allarme` non ha scritto `commuta`: perché `casa[2].commuta()` funziona?
5. se togliessimo `isAcceso` da `Lampada`, quale errore darebbe il compilatore, e su quale classe?

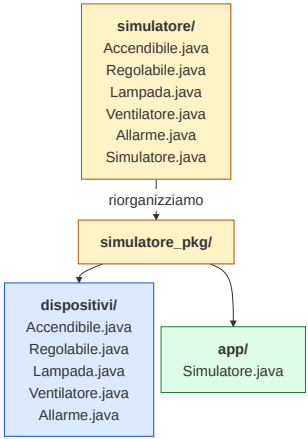
Insieme, passo passo (30 min) – il problema

Il problema. Il simulatore cresce: altri dispositivi, altri programmi che li usano. Le due interfacce e le tre classi dei dispositivi sono **riusabili**, il `Simulatore` no: è uno dei tanti programmi possibili. Vogliamo **riorganizzare il progetto in package**: i dispositivi formano una piccola libreria, separata dai programmi che la usano.

Le regole

- il codice delle classi non cambia: cambiano la posizione dei file e le righe iniziali
- un package `dispositivi` per le interfacce e le classi dei dispositivi, un package `app` per il `Simulatore`
- tutto ciò che `app` usa deve essere **visibile** da un altro package
- alla fine il programma si compila e si esegue dalla radice e stampa quello di prima

Partiamo dalla cartella `simulatore/` con i sei file. Un passo alla volta, compilando dal terminale per vedere **gli errori** che ogni passo produce finché il successivo non lo completa.



Insieme, passo passo – le decisioni

Prima di toccare i file, decidiamo insieme:

1. **quali package?** → `dispositivi`: le due interfacce e le tre classi, coese fra loro e senza dipendenze dal programma; `app`: il solo `Simulatore`, che dipende da `dispositivi` e non viceversa
2. **le cartelle?** → una radice `simulatore_pkg/` con dentro `dispositivi/` e `app/`: stesso nome dei package, lettera per lettera
3. **la prima riga di ogni file?** → `package dispositivi;` nei cinque file dei dispositivi, `package app;` in `Simulatore.java`
4. `Simulatore` **come trova le classi?** → con gli `import`: uno per ogni nome che usa (`Accendibile`, `Regolabile`, `Lampada`, `Ventilatore`, `Allarme`), oppure `import dispositivi.*;`
5. **che cosa deve essere** `public`**?** → tutto ciò che `app` nomina: le due interfacce e le tre classi, con i loro costruttori. Le interfacce e `Lampada`, `Ventilatore` lo sono già; `Allarme` **no**: il compilatore ce lo dirà
6. **da dove si compila?** → dalla radice: `javac app/Simulatore.java`, poi `java app.Simulatore`

Gli errori che si incontrano

- file spostati in `dispositivi/`, `Simulatore` ancora senza `import`: *"cannot find symbol: class Lampada"*
- `import` messi, ma in un file manca la riga `package`: *"cannot access Accendibile: bad source file . . . file does not contain class dispositivi.Accendibile"*
- `import dispositivi.Allarme;` con `Allarme` senza `public`: *"Allarme is not public in dispositivi; cannot be accessed from outside package"*
- `java Simulatore` da dentro `app/`: *"Could not find or load main class Simulatore"*

Ogni messaggio dice esattamente qual è il passo che manca.

Una possibile soluzione (1/2) – il package `dispositivi`

`dispositivi/Accendibile.java`, prime righe:

```
package dispositivi;

/** Un dispositivo: si accende e spegne. */
public interface Accendibile {
```

`dispositivi/Regolabile.java`:

```
package dispositivi;

/** Ha un livello regolabile a passi. */
public interface Regolabile {
```

`dispositivi/Lampada.java` (e così `Ventilatore.java`):

```
package dispositivi;

/** Una lampada: luce regolabile da 1 a 10. */
public class Lampada
        implements Accendibile, Regolabile {
```

`dispositivi/Allarme.java`, ora `public`:

```
package dispositivi;

/** Un allarme: acceso o spento. */
public class Allarme implements Accendibile {
```

- il resto dei cinque file è **identico** alla versione di prima: il codice non sa in che package sta
- `Lampada` usa `Accendibile` e `Regolabile` senza `import`: stesso package
- `Allarme` senza `public` compilava da sola e con le altre: `dispositivi` la vede. È app che non la vedeva: la visibilità di package, finora invisibile, qui conta. Il suo costruttore era già `public`: anche lui deve esserlo, altrimenti `new Allarme()` da app fallisce con un messaggio analogo, sul costruttore
- nessun `import` di `java.lang`: `String` e `System` si vedono sempre

Una possibile soluzione (2/2) – il package app

app/Simulatore.java, prime righe:

```
package app;

import dispositivi.Accendibile;
import dispositivi.Regolabile;
import dispositivi.Lampada;
import dispositivi.Ventilatore;
import dispositivi.Allarme;

/** Dispositivi diversi, tutti Accendibile. */
public class Simulatore {

    public static void main(String[] args) {
        Accendibile[] casa = {
            new Lampada("cucina"),
```

Il `main` prosegue come prima, senza una modifica. Cinque `import` o uno solo, `import dispositivi.*;`: con i nomi espliciti si legge subito da dove viene ogni classe.

Dalla radice `simulatore_pkg/`:

```
$ javac app/Simulatore.java
$ java app.Simulatore
lampada cucina: accesa, luce 8/10
ventilatore studio: acceso, velocita 3/3
allarme: disattivo
lampada camera: spenta
accesi: 2
```

- `javac` compila `Simulatore` e, seguendo gli `import`, i cinque file di `dispositivi/`: i `.class` compaiono nelle due cartelle
- `java app.Simulatore`: nome qualificato, dalla radice
- stesso output di prima: la riorganizzazione **non** cambia il comportamento, cambia la struttura. Un nuovo programma in `app/` riusa i dispositivi con gli stessi `import`; un nuovo dispositivo va in `dispositivi/`, `public`, con la sua riga `package`: è l'homework

Riepilogo: cosa abbiamo trattato

Concetto (unità)	Dove compare oggi
`interface`: solo intestazioni e costanti; quando usarla (U17)	`Pagabile`; `Accendibile`, `Regolabile`
`implements`, metodi `public`, errore se ne manca uno (U17)	`Fattura`, `Dipendente`; `Lampada`, `Ventilatore`, `Allarme`
L'interfaccia come tipo: polimorfismo, `instanceof`, niente `new` (U17)	`Pagabile[] daPagare`; `Accendibile[] casa`, `instanceof Regolabile`
Più interfacce per una classe; un'interfaccia ne estende un'altra (U17)	`Fattura implements Rateizzabile, Stampabile`; `Rateizzabile extends Pagabile`
Metodi `default` e `static` (U17)	`rata()`, `Pagabile.totale`; `commuta()`, `Accendibile.quantiAccesi`
Interfacce o classi astratte; `Cloneable` (U17)	la tabella; `implements Cloneable` della lezione 8
`package`, nome qualificato, cartelle, package di default (U18)	`contabilita_pkg`; `simulatore_pkg`
Compilare ed eseguire dalla radice; `import`; `java.lang` (U18)	`javac app/Simulatore.java`; `java app.Simulatore`; `app/ProvaPagabili`
Visibilità di package e riepilogo dei modificatori (U18)	la tabella; `Allarme` senza `public`

Prossima lezione (L10): le eccezioni – `try/catch/finally`, lo stack trace, `throws` (finalmente `CloneNotSupportedException`), eccezioni controllate e non, eccezioni nostre; la ricorsione – casi base e ricorsivi, lo stack delle chiamate, da ricorsivo a iterativo.

Homework 9 (da svolgere entro la prossima lezione)

Tre esercizi, ciascuno con il suo programma di prova; il secondo e il terzo usano i package.

1. `Spedibile.java` (con `Pacco.java` e `Lettera.java`): un'interfaccia con la costante `TARIFFA_AL_KG` $= 2.0$, il metodo `peso()` (in kg) e il metodo `default costoSpedizione()` $= 3.0 +$ tariffa $\times$ peso. `Pacco` (destinatario, peso) usa il costo `default`; `Lettera` (destinatario, grammi `int`) converte in kg e **ridefinisce** `costoSpedizione()`: 1.20 fisso. `ProvaSpedizioni`: uno `Spedibile[]` con due pacchi e due lettere; stampa il costo di ciascuno (`printf`, due decimali), il totale con un suo metodo `static costoTotale(Spedibile[])`, quante lettere ci sono (`instanceof`), la costante.
2. `Termostato.java`: copiate la cartella `simulatore_pkg` di oggi in `homework/termostato/`. Nel package `dispositivi` aggiungete `Termostato`, che implementa `Accendibile` e `Regolabile`: acceso o spento, temperatura `int` da 15 a 30 gradi (parte da 20), `alza`/`abbassa` di un grado solo se acceso; `toString` come `termostato: acceso, 22 gradi` (o `spento, 20 gradi`). In app, `ProvaTermostato`: un `Accendibile[]` con un termostato e una lampada; alza il termostato da spento (non cambia), accende tutti, alza due volte i regolabili con `instanceof`, stampa il termostato e il suo livello, lo commuta e ristampa, conta gli accesi con `quantiAccesi`.
3. `Magazzino.java` (package `magazzino`, con `Articolo.java` e `Scorta.java`): `Articolo` è `public` e immutabile (codice, nome, prezzo; `toString`; `equals` e `hashCode` sul solo codice). `Scorta` è **senza** `public`, come il costruttore e i metodi (tranne `toString`); variabili `private`: un articolo e la quantità; `aggiungi(int n)`, `togli(int n)` (`false` se i pezzi non bastano), `valore()`. `Magazzino` è `public`: uno `Scorta[]` con la capienza data al costruttore; `carica(Articolo a, int n)` aggiunge alla scorta di a se c'è (cercata con `equals`), altrimenti la crea (`false` se non c'è posto); `scarica(Articolo a, int n)`; `disponibili(Articolo a)`; `valoreTotale()`; `toString`. `app.ProvaMagazzino` carica due articoli, ricarica il primo con un `Articolo` nuovo con lo stesso codice, scarica più del disponibile e poi meno, stampa disponibilità, valore e magazzino; prova `new magazzino.Scorta(...)`: leggete l'errore, poi commentate la riga.

Domande?

mattia.fonisto@unina.it